

como incluir tools para acesso ao gmail?



Thought for 2.17s ✓

Clarifying Gmail tools

I need to provide my answer in Portuguese since the user asked about including tools for Gmail access. They're likely looking for information on integrating this into the ChatGPT agent or API. I'll clarify the contexts and mention options regarding security. I could explain OAuth, the Gmail API, and maybe the MCP as well. I should also ask what platform they're using at the end, but I'll aim for a one-shot response.

Done ✓

Para incluir ferramentas com acesso ao Gmail, o caminho recomendado é usar a **Gmail API com OAuth 2.0**. Evite pedir a senha do usuário ou usar credenciais SMTP diretamente.

1. Criar um projeto no Google Cloud

1. Acesse o console.cloud.google.com.
2. Crie ou selecione um projeto.
3. Ative a **Gmail API**.
4. Configure a tela de consentimento OAuth.
5. Crie credenciais do tipo **OAuth Client ID**.
6. Escolha o tipo de aplicação:
 - Aplicação web
 - Desktop
 - Serviço no servidor

2. Definir os escopos necessários

Use somente as permissões indispensáveis:

text	Collapse	Copy
<code>https://www.googleapis.com/auth/gmail.readonly</code>		

Para enviar mensagens:

text	Collapse	Copy
<code>https://www.googleapis.com/auth/gmail.send</code>		

Para modificar, arquivar ou excluir mensagens:

text	Collapse	Copy
<code>https://www.googleapis.com/auth/gmail.modify</code>		

Evite usar:

text	Collapse	Copy
<code>https://mail.google.com/</code>		

Esse escopo concede acesso total à conta.

3. Exemplo em Python

Instale as bibliotecas:

```
bash
```

```
pip install --upgrade google-api-python-client google-auth-httpplib2 google-auth-oauthlib
```

Código básico:

```
python
```

```
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build

SCOPES = [
    "https://www.googleapis.com/auth/gmail.readonly"
]

flow = InstalledAppFlow.from_client_secrets_file(
    "credentials.json",
    SCOPES
)

credentials = flow.run_local_server(port=0)

gmail = build("gmail", "v1", credentials=credentials)

resultado = gmail.users().messages().list(
    userId="me",
    maxResults=10
).execute()

for mensagem in resultado.get("messages", []):
    print(mensagem["id"])
```

Na primeira execução, o usuário autoriza o acesso no navegador. Depois, você pode armazenar o token OAuth com segurança.

4. Transformar isso em uma tool para um agente

A tool pode ter uma interface como:

```
json
```

```
{
  "name": "listar_emails",
  "description": "Lista emails recentes da caixa de entrada",
  "parameters": {
    "type": "object",
    "properties": {
      "quantidade": {
        "type": "integer",
        "description": "Número máximo de emails"
      }
    }
  }
}
```

```
}
},
"required": []
}
}
```

No backend, a implementação chama:

python

Collapse

Copy

```
gmail.users().messages().list(
    userId="me",
    labelIds=["INBOX"],
    maxResults=quantidade
).execute()
```

Outras tools úteis:

- buscar_emails
- ler_email
- listar_anexos
- enviar_email
- responder_email
- arquivar_email
- marcar_como_lido

5. Cuidados importantes

- Armazene tokens criptografados.
- Nunca exponha `client_secret.json` no frontend.
- Exija confirmação antes de enviar, excluir ou alterar emails.
- Restrinja o acesso por usuário e conta.
- Registre auditoria das ações executadas.
- Solicite apenas os escopos OAuth necessários.
- Para uso corporativo, considere políticas do Google Workspace e verificação do aplicativo.

Se você estiver usando **OpenAI Assistants/Responses API, GPTs com Actions, LangChain, n8n, Make, Zapier ou MCP**, a forma de incluir a tool muda um pouco, mas em todos os casos o acesso deve ser feito por OAuth e por um backend seguro.

exporte as instruções acima em arquivo mark down



Salve o conteúdo abaixo como **acesso-gmail-tools.md**:

markdown

Collapse

Copy

Como incluir tools para acesso ao Gmail

O caminho recomendado para incluir ferramentas com acesso ao Gmail é usar a **Gmail API**.

Evite solicitar a senha do usuário ou usar credenciais SMTP diretamente.

1. Criar um projeto no Google Cloud

1. Acesse o [Google Cloud Console](https://console.cloud.google.com/).
2. Crie ou selecione um projeto.

3. Ative a **Gmail API**.
4. Configure a tela de consentimento OAuth.
5. Crie credenciais do tipo **OAuth Client ID**.
6. Escolha o tipo de aplicação:
 - Aplicação web
 - Aplicação desktop
 - Serviço no servidor

2. Definir os escopos necessários

Use somente as permissões indispensáveis.

Ler emails

```
```text
https://www.googleapis.com/auth/gmail.readonly
```

## Enviar emails

text

Collapse

Copy

```
https://www.googleapis.com/auth/gmail.send
```

## Modificar emails

Esse escopo permite arquivar, marcar como lido e realizar outras alterações:

text

Collapse

Copy

```
https://www.googleapis.com/auth/gmail.modify
```

Evite usar o escopo abaixo, pois ele concede acesso total à conta:

text

Collapse

Copy

```
https://mail.google.com/
```

## 3. Exemplo em Python

Instale as bibliotecas necessárias:

bash

Collapse

Copy

```
pip install --upgrade google-api-python-client google-auth-http lib2 google-auth-o
```

Exemplo básico:

python

Collapse

Copy

```
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build
```

```

SCOPES = [
 "https://www.googleapis.com/auth/gmail.readonly"
]

flow = InstalledAppFlow.from_client_secrets_file(
 "credentials.json",
 SCOPES
)

credentials = flow.run_local_server(port=0)

gmail = build("gmail", "v1", credentials=credentials)

resultado = gmail.users().messages().list(
 userId="me",
 maxResults=10
).execute()

for mensagem in resultado.get("messages", []):
 print(mensagem["id"])

```

Na primeira execução, o usuário autoriza o acesso no navegador. Depois, o token OAuth pode ser armazenado com segurança para evitar uma nova autorização a cada execução.

## 4. Transformar a integração em uma tool

Uma tool pode ser definida com uma interface semelhante à seguinte:

json

Collapse

Copy

```

{
 "name": "listar_emails",
 "description": "Lista emails recentes da caixa de entrada",
 "parameters": {
 "type": "object",
 "properties": {
 "quantidade": {
 "type": "integer",
 "description": "Número máximo de emails"
 }
 }
 },
 "required": []
}

```

No backend, a implementação pode chamar a Gmail API:

python

Collapse

Copy

```

gmail.users().messages().list(
 userId="me",
 labelIds=["INBOX"],

```

```
maxResults=quantidade
).execute()
```

## 5. Exemplos de tools úteis

- `listar_emails`
- `buscar_emails`
- `ler_email`
- `listar_anexos`
- `enviar_email`
- `responder_email`
- `arquivar_email`
- `marcar_como_lido`

## 6. Cuidados importantes

- Armazene os tokens OAuth de forma criptografada.
- Nunca exponha o arquivo `client_secret.json` no frontend.
- Exija confirmação antes de enviar, excluir ou alterar emails.
- Restrinja o acesso por usuário e por conta.
- Registre uma auditoria das ações executadas.
- Solicite somente os escopos OAuth necessários.
- Para uso corporativo, considere as políticas do Google Workspace.
- Verifique se o aplicativo precisa passar pelo processo de verificação do Google.

## 7. Compatibilidade com outras plataformas

A forma de incluir a tool varia conforme a plataforma utilizada:

- OpenAI Responses API
- OpenAI Assistants
- GPTs com Actions
- LangChain
- n8n
- Make
- Zapier
- MCP

Em todos os casos, o acesso ao Gmail deve ser implementado usando **OAuth 2.0** e um backend seguro.

...